



DB ADMIN PORTAL

Role-Based Database Management System with AI Data Analyst

Developed By

Kashif , Anusha, Abhay, Akash, Abhinay, Raj , Rahul, Shristi and Vaishnavi

Python

Streamlit

SQLAlchemy ORM

MySQL

OpenRouter AI

Existing Problems

Why traditional database access falls short for most teams

-  Managing databases requires SQL knowledge.
-  No centralized user management.
-  Difficult permission control.
-  Data modifications are not tracked.
-  Business users cannot analyze data without SQL.
-  High chances of accidental data modification.

Our Solution

A secure Database Administration Portal that provides:



Secure Login



Role-Based Access



Dynamic CRUD



User Management



Permission Management



Audit Logs



AI Data Analysis using Natural Language

Objectives

What we set out to build

- ① Build a secure database management portal
- ① Implement Role-Based Access Control
- ① Create generic CRUD operations
- ① Dynamically reflect database schema
- ① Track every database activity
- ① Integrate AI for data analysis

Technology Stack



Python 3.11

Backend



Streamlit

Web Interface



SQLAlchemy ORM

ORM Layer



MySQL

Database



Pandas

Data Processing



bcrypt

Password Encryption



Requests

API Communication



OpenRouter

AI Integration

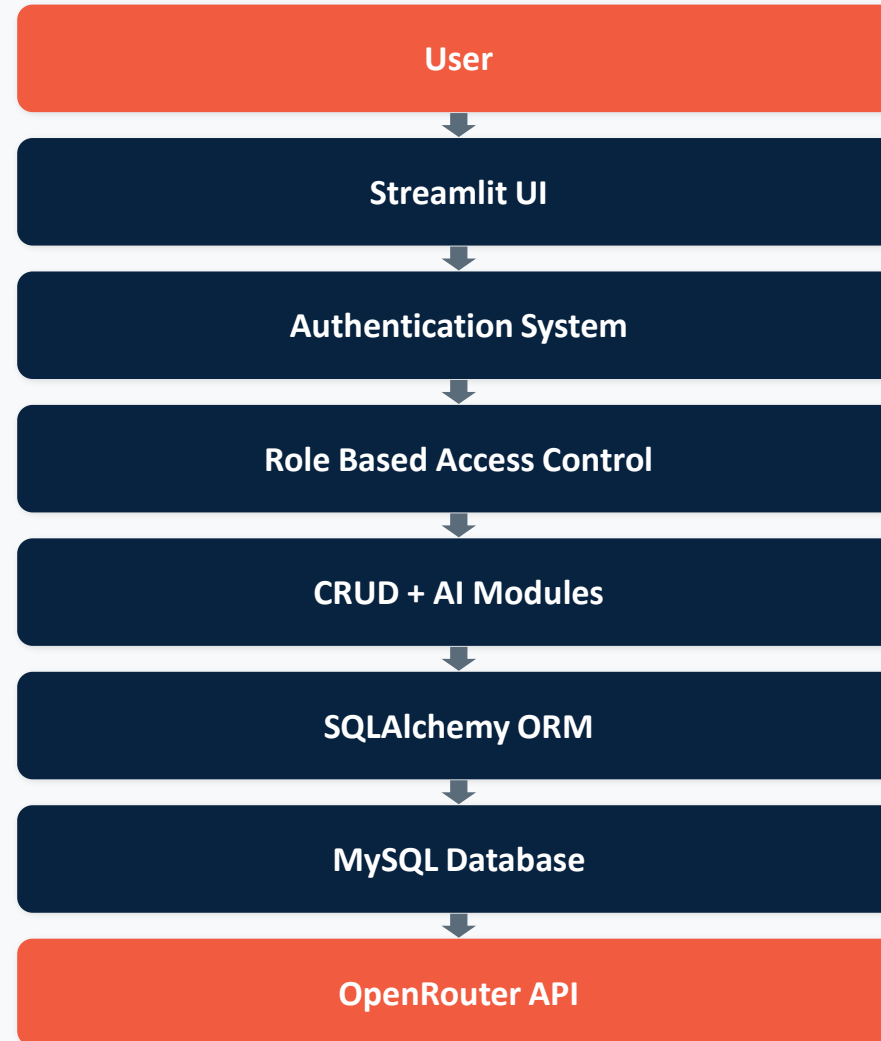
Project Team

Eight members,owning a distinct part of the system

Role	Team Member	Responsibilities
Team Lead and Management	Anusha & Kashif	Project Planning, Architecture, AI Integration, Code Integration, GitHub repository management, code review, testing, and team coordination.
Authentication Developer	Kashif & Akash	Developed the login system, session management, password hashing using bcrypt, and user authentication flow.
Database Developer	Anusha & Abhay	Designed the MySQL database schema, created ORM models, configured database connections, and implemented SQLAlchemy reflection.
CRUD Developer	Abhinay , Rahul and Raj	Implemented dynamic Create, Read, Update, and Delete operations with automatic form generation.
Permission and Audit Developer	Kashif & Akash	Developed RBAC, user management, permissions, and audit logging
UI Developer	Vaishnavi , Shristi and Kashif	Designed Streamlit pages, dashboard layout, navigation menu, and improved the user interface.
AI Integration	Kashif & Abhinay	Integrated AI chatbot, SQL generation, validation, and data analysis using OpenRouter API.

System Architecture

A layered flow from the user down to the AI-powered database layer



Database Design

Core tables powering the portal

base.py

- Creates the common **Declarative Base** class
- Parent class for all ORM models
- Enables Object Relational Mapping (ORM)

session.py

- Creates reusable SQLAlchemy sessions using SessionLocal
- Manages database interaction efficiently
- Connects application with the MySQL database engine

models.py

Defines Database Schema using ORM Models

- **User** – Stores login credentials, roles and account status
- **TablePermission** – Manages table-level CRUD permissions for each user
- **AuditLog** – Records every important database activity with timestamps and status
- **Sample tables** for business data

```
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

```
from sqlalchemy.orm import sessionmaker

from database.connection import engine

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)
```

```
mysql> describe users;
```

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI	NULL	auto_increment
username	varchar(100)	NO	UNI	NULL	
password	varchar(255)	NO		NULL	
role	varchar(20)	NO		NULL	
is_active	tinyint(1)	YES		NULL	
created_at	datetime	YES		now()	DEFAULT_GENERATED

```
6 rows in set (0.00 sec)

mysql> select * from users;
```

user_id	username	password	role	is_active	created_at
3	abhinay	\$2b\$12\$0paqEnFeLrgIyCr3G2Js00rProPE/40UpRLTt8yPjJJuNIwdDIzQm	viewer	1	2026-04-01 12:00:00
5	Admin	\$2b\$12\$xbg1KHVvNaHvpLVFNgrLuENrgeE83bmDfe9U1zMTIO.qJ3/Sjd.R	admin	1	2026-04-01 12:00:00
6	abhay	\$2b\$12\$Jwy8ki/vou/a1R0yzzDwaeFWQrr931.BNGHwx.uAp9.LoLbGncvr.	admin	1	2026-04-01 12:00:00
7	Aakash	\$2b\$12\$dmkt1qgaF/jERb9uGuxruOnWB3XYLU0kI/Ae1rpfpcT/93SfgwgG	editor	1	2026-04-01 12:00:00
8	Shiva	\$2b\$12\$7KXy6a4RRh8yswXeJYhUCe486RovMQ78F1QffCp7u4z13lbn1GkTra	editor	1	2026-04-01 12:00:00
9	Amaan	\$2b\$12\$qa5Qa8QfB9kjUfmowheUdu1FOTyXN7sqk28gU6wU9ABgnZu6a.9H2	editor	1	2026-04-01 12:00:00

```
6 rows in set (0.00 sec)
```


Database Design

Core tables powering the portal

connection.py

- Creates the SQLAlchemy Engine
- Establishes secure connection with MySQL
- Centralizes database configuration

reflection.py

- Dynamically reflects existing database schema
- Retrieves tables, columns and primary keys
- Eliminates hardcoded database structures

create_admin.py

- Creates the default **Admin** account
- Initializes the application with administrator credentials
- Ensures secure first-time access to the portal

Key Contributions:

- ✓ Configured MySQL database connectivity
- ✓ Implemented dynamic schema reflection
- ✓ Automated admin account initialization
- ✓ Enabled flexible and scalable database operations

Authentication Module

Secure login built on hashed credentials and session state



Login



Password Hashing



Session Management



Role Verification

Role Based Access Control

What each role can and cannot do

Feature	Admin	Editor	Viewer
View Data	✓	✓	✓
Insert	✓	✓	✗
Update	✓	✓	✗
Delete	✓	✗	✗
User Management	✓	✗	✗
Permission Management	✓	✗	✗
Audit Logs	✓	✗	✗
AI Analysis	✓	✓	✓

Dynamic Database Reflection

No hardcoded tables — the portal adapts to your schema automatically

Implemented Functions

-  Get All Tables
-  Get Table Object
-  Get Primary Key
-  Dynamic Schema Detection

Benefits

-  No hardcoded tables
-  Generic CRUD
-  Supports new tables automatically

CRUD Operations

Generic operations that work on any reflected table



Create

Dynamic Forms

screenshot



Read

Dynamic Data Viewer

screenshot



Update

Load Record

screenshot



Delete

Delete by Primary Key

screenshot

User & Permission Management

What an Admin can do



Create Users



Delete Users



Activate / Deactivate Users



Assign Permissions



Table-Level Permissions

Permissions aren't just role-wide — an Admin can grant or revoke access to specific tables for a specific user, so an Editor can be scoped to only the tables they should touch.

Audit Logging

A complete, tamper-evident trail of activity

Tracks

-  Login
-  Insert
-  Update
-  Delete

Benefits

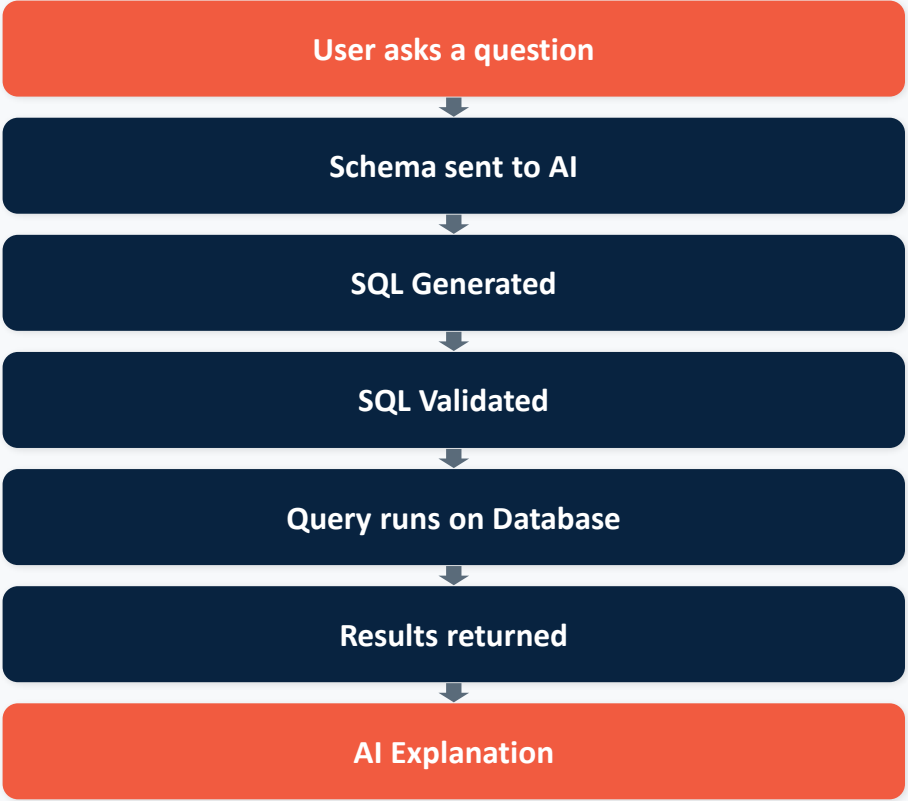
- Security
- Accountability
- Monitoring

Audit Logs

	ID	User	Action	Table	Status	Error	Timestamp
0	20	admin	LOGIN		SUCCESS		2026-07-04 12:42:32
1	19	aakash	LOGIN		SUCCESS		2026-06-16 03:45:17
2	18	admin	LOGOUT		SUCCESS		2026-06-16 03:45:08
3	17	admin	CREATE_USER	users	SUCCESS		2026-06-16 03:44:36
4	16	admin	TOGGLE_USER_STATUS	users	SUCCESS		2026-06-16 03:43:35
5	15	admin	TOGGLE_USER_STATUS	users	SUCCESS		2026-06-16 03:43:33
6	14	admin	TOGGLE_USER_STATUS	users	SUCCESS		2026-06-16 03:43:30
7	13	admin	TOGGLE_USER_STATUS	users	SUCCESS		2026-06-16 03:43:28
8	12	admin	TOGGLE_USER_STATUS	users	SUCCESS		2026-06-16 03:43:26
9	11	admin	TOGGLE_USER_STATUS	users	SUCCESS		2026-06-16 03:43:22

AI Data Analyst

Ask questions in plain English — get validated SQL and real answers



AI Security

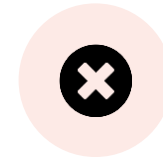
Every AI-generated query passes through a strict SQL validator



Only Allows

SELECT

Read-only access ensures the AI can analyze data but never change it.



Blocks

INSERT • UPDATE • DELETE • DROP •
ALTER • TRUNCATE • CREATE

Live Example

User Question

“Show top 5 highest salary employees”

Generated SQL

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

AI Explanation

The query retrieves all employee records sorted by salary in descending order and returns only the top 5, directly answering the user's question.

Output Table

Name	Department	Salary
Employee A	Engineering	₹ 1,85,000
Employee B	Sales	₹ 1,72,000
Employee C	Engineering	₹ 1,68,000
Employee D	Finance	₹ 1,60,000
Employee E	Marketing	₹ 1,55,000

Sample output shown for illustration — replace with a real screenshot for the actual demo.

Challenges & Solutions

Real engineering problems we hit — and how we solved them

Challenge	Solution
SQLAlchemy Reflection	Dynamic Reflection Functions
Dynamic CRUD	Generic CRUD Modules
TINYINT Boolean Handling	Checkbox Mapping
Session Management	Streamlit Session State
AI Integration	OpenRouter API
SQL Security	SQL Validator
SSL Issues	Certificate Configuration

Future Enhancements

Where we're taking this next



Excel Export



CSV Export



Dashboard Analytics



Charts & Graphs



Bulk Upload



Backup & Restore



Multi-Database Support



Voice-based AI Queries



AI Recommendations



Cloud Deployment

Conclusion & Demo

Project Summary

- ✓ Secure Authentication
- ✓ Role-Based Access Control
- ✓ Dynamic Database Reflection
- ✓ Generic CRUD Operations
- ✓ User & Permission Management
- ✓ Audit Logging
- ✓ AI-Powered Natural Language Data Analysis (In Progress)

Live Demo Flow

- | | | | |
|---|-----------------|----|--------------------|
| 1 | Login | 2 | Dashboard |
| 3 | View Tables | 4 | Insert Record |
| 5 | Update Record | 6 | Delete Record |
| 7 | Manage Users | 8 | Assign Permissions |
| 9 | View Audit Logs | 10 | AI Data Analyst |



Thank You

Questions & Answers

Coforge